PROJETO DIDÁTICO | VERSÃO 0.1

Mini-8

Processador de 8 bits
ISA, caminho dos dados, matriz de controle e simulação

Seminário MIPS - 03/11/2026

PPCA / UFRR | Prof. Herbert Oliveira Rocha
Proposta elaborada em 02/11/2026 para avaliação antes da implementação física.

Objetivo
Construir uma CPU pequena o suficiente para acompanhar cada bit: carregar dados, somar, armazenar resultados e executar desvios condicionais. Esta ISA é própria; não executa código de máquina MIPS.

1. Base da proposta

O slide 14 pede datapath, matriz de controle e simulação e apresenta, como exemplo, o particionamento 2+3+3. Adotamos esse exemplo. As instruções, capacidades de memória, comportamento de R0 e regras de execução abaixo são decisões desta proposta, não especificações adicionais atribuídas ao professor. [1]

Elemento	Escolha da versão 0.1
Tamanho da instrução	8 bits: 2 de opcode + 3 de A + 3 de B
Dados e registradores	8 bits; soma com resultado limitado a 8 bits
Banco de registradores	R0 a R7: R0 fixo em zero; R1 a R7 graváveis
PC e endereços	8 bits: posições de 0 a 255
Memórias separadas	ROM: 256 bytes de instruções; RAM: 256 bytes de dados
Quatro operações	ADD, LD, ST e BEQZ
Modelo de execução	Uniciclo, com leituras combinacionais e escritas na borda

Duas escolhas distintas: a instrução tem 8 bits e, por decisão nossa, o caminho de dados também tem 8 bits. Uma escolha não determina automaticamente a outra.

2. Como os 8 bits viram instruções

```
bits:    7 6       5 4 3       2 1 0
campo: [ opcode ] [   A   ]   [   B   ]
           2         3           3       = 8 bits
```

A e B são nomes neutros para os campos. A função de cada campo depende do opcode. O programa escreve primeiro o destino em ADD e LD, embora o destino esteja no campo B da codificação.

Op.	Sintaxe	Campo A	Campo B	Efeito
00	ADD Rd,Rs	Rs	Rd	R[d] ← R[d] + R[s]
01	LD Rd,[Ra]	Ra (base)	Rd	R[d] ← M[R[a]]
10	ST Rs,[Ra]	Ra (base)	Rs	M[R[a]] ← R[s]
11	BEQZ Rn,off	Rn	off com sinal	Se R[n]=0, PC ← PC+1+off

2.1. Uma soma completa em um byte

Mantemos a codificação usada durante nossa aula: ADD R5,R2 soma o valor de R2 ao valor de R5 e guarda em R5.

```
ADD R5,R2  ->  00 | 010 | 101  ->  00010101  ->  0x15
                  R2    R5
R2 = 10, R5 = 20  ->  R5 passa a 30
```

2.2. Deslocamento do desvio

Somente BEQZ usa B como número imediato. O deslocamento tem 3 bits em complemento de dois e é contado a partir de PC+1, em bytes. Como cada instrução ocupa um byte, +1 pula uma instrução e -1 retorna para a própria instrução de desvio.

B	000	001	010	011	100	101	110	111
Valor	0	+1	+2	+3	-4	-3	-2	-1

```
BEQZ R0,-1  ->  11 | 000 | 111  ->  0xC7
PC seguinte = PC + 1 - 1 = PC
```

Limites deliberados
LD e ST transferem um byte, sem deslocamento: o registrador-base já contém o endereço. Não são lw/sw do MIPS. BEQZ testa um registrador contra zero, não dois registradores independentes. Não existe HALT: 0xC7 apenas forma um laço inerte.

3. Caminho dos dados (datapath)

O desenho separa o endereço de memória, o dado a armazenar e o valor de retorno. Todos os caminhos de dados mostrados têm 8 bits; A e B são índices de registrador com 3 bits.

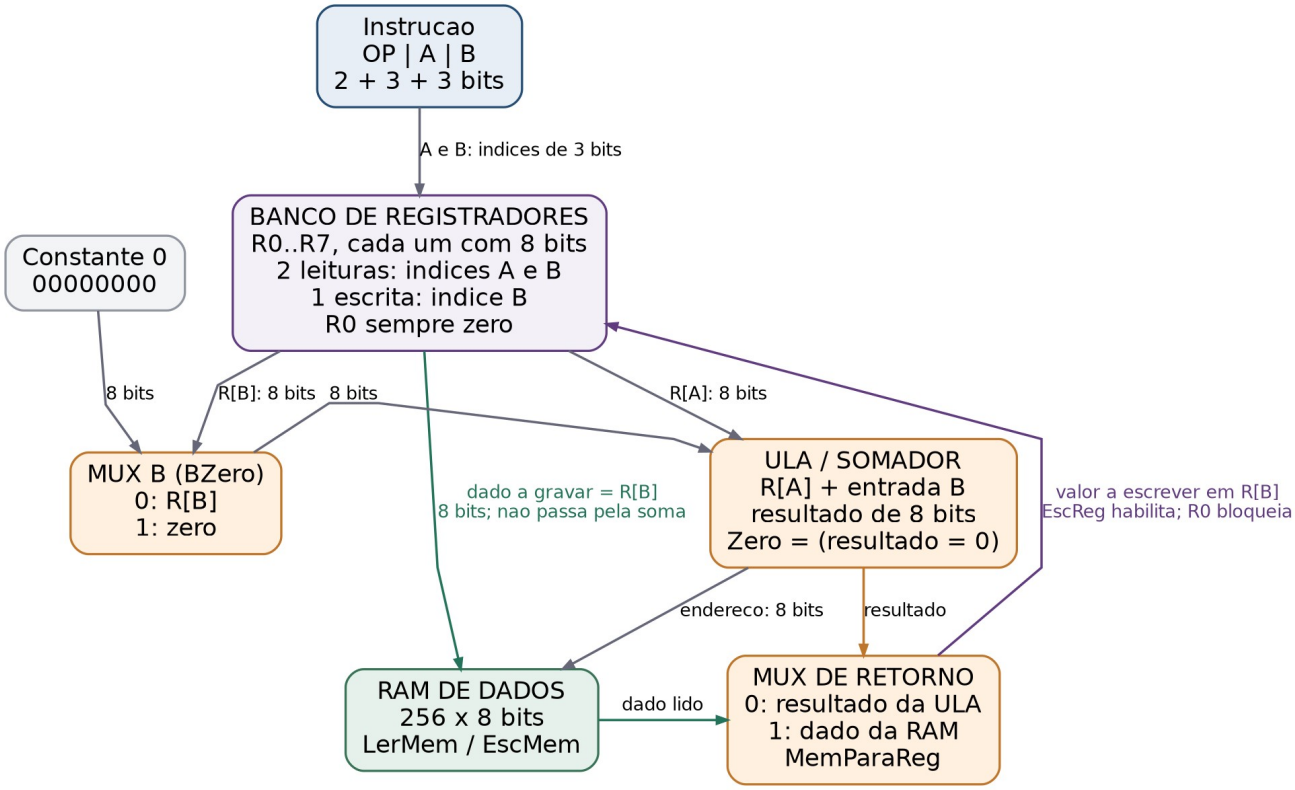


Figura 1. Datapath de dados da proposta. PC/ROM e decisão de desvio aparecem na Figura 2. Os sinais de controle são definidos integralmente na matriz da página seguinte.

Componente / ligação	Contrato desta versão
Banco de registradores	Lê R[A] e R[B]; grava somente em B. Escritas em R0 são descartadas.
MUX B / BZero	0 seleciona R[B]. 1 seleciona a constante zero.
ULA	Sempre soma. Em ADD: R[A]+R[B]. Nas demais: R[A]+0.
Memória de dados	Recebe o resultado da ULA como endereço; ST envia R[B] separadamente como dado.
MUX MemParaReg	Escolhe o resultado da ULA (0) ou o byte lido da RAM (1).
Extensão de sinal	O off3 de BEQZ é estendido a 8 bits no circuito do PC, não na entrada da ULA de dados.

3.1. Estado e borda de clock

Durante o ciclo, leituras e cálculos são combinacionais. Na borda de subida, PC, registrador de destino e eventual escrita de RAM são atualizados conforme a instrução. Não há um registrador de instrução extra nem estágios de pipeline.

Reset síncrono ativo em 1 zera PC e registradores, mas preserva a RAM. enable=0 impede avanço do PC, escrita nos registradores e escrita na RAM. A carga inicial da RAM pertence ao ambiente de simulação/integração, não a uma instrução escondida.

4. Matriz de controle e desvio

Matriz própria do Mini-8, inspirada na organização estudada nos slides 6 a 9. Não é a matriz original do MIPS. [1]

Sinal	ADD	LD	ST	BEQZ
BZero	0	1	1	1
EscReg	1	1	0	0
LerMem	0	1	0	0
EscMem	0	0	1	0
MemParaReg	0	1	X	X
DvC	0	0	0	1

BZero é o seletor da segunda entrada da ULA. Usamos um nome diferente de ULAFonte porque aqui a alternativa é zero, não o imediato. X significa irrelevante; a implementação gera 0 nesses casos. EscReg ainda é bloqueado quando B=0, enable=0 ou reset=1.

```
Zero = 1 quando o resultado de 8 bits da ULA vale zero
FontePC = DvC AND Zero
PC_sequencial = (PC + 1) mod 256
PC_desvio = (PC + 1 + sext3(B)) mod 256
PC_proximo = PC_desvio se FontePC=1; senao PC_sequencial
```

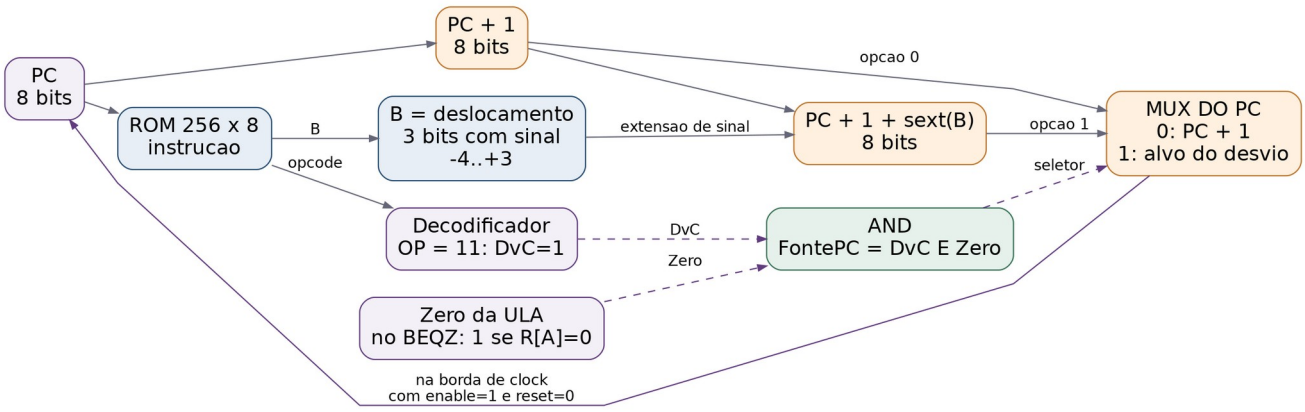


Figura 2. Busca, atualização do PC e decisão de desvio. A saída Zero mostrada vem da ULA da Figura 1. O valor do PC só é atualizado na borda habilitada.

No BEQZ, a ULA calcula R[A]+0; portanto Zero=1 indica R[A]=0. Um ADD que resulte em zero não desvia, porque DvC=0. Zero é combinacional: não existe um registrador de flags persistente.

Não precisamos de funct nem de um segundo decodificador ULAOp: há quatro opcodes completos e uma única operação aritmética. RegDst também desaparece: toda escrita em registrador usa B.

5. Programa 1: carregar, somar e guardar

Objetivo: calcular 2 + 3 e gravar 5 na RAM. Na partida, todos os registradores estão zerados; M[0]=2 e M[2]=3. As demais posições de dados valem zero. O resultado substituirá intencionalmente o operando armazenado em M[2].

PC	Hex	Instrução	O que ocorre
0	41	LD R1,[R0]	R1 recebe M[0] = 2
1	0A	ADD R2,R1	R2 recebe 0+2 = 2: prepara o endereço
2	53	LD R3,[R2]	R3 recebe M[2] = 3
3	0C	ADD R4,R1	R4 recebe 0+2 = 2
4	1C	ADD R4,R3	R4 recebe 2+3 = 5
5	94	ST R4,[R2]	M[2] recebe R4 = 5
6	C7	BEQZ R0,-1	R0=0: volta ao PC 6; não para o clock

Resultado realmente obtido no simulador
R0=0; R1=2; R2=2; R3=3; R4=5; R5=R6=R7=0.
RAM[2]=5. PC=6 no laço final. Foram simulados 7 ciclos habilitados: 6 de trabalho e 1 iteração do laço final.

5.1. O que esse exemplo comprova

O modelo carrega operandos da RAM, constrói um endereço em registrador, usa uma soma de dois operandos, grava o resultado na RAM e executa um desvio. O programa não usa inicialização secreta de registradores nem uma quinta instrução para carregar constantes.

5.2. Como reproduzir

```
python3 src/mini8.py examples/soma.asm \
  --data examples/soma_data.json --steps 7 \
  --csv results/soma_trace.csv
```

Execute o comando na pasta principal do projeto. O montador aceita rótulos e gera bytes reais; o simulador registra o PC, opcode, operandos, sinais, efeito e registradores após cada passo. No Windows, pode ser necessário usar py -3 no lugar de python3.

Arquivos de evidência: results/soma_execucao.txt, results/soma_trace.csv e results/soma_rom.hex. Os valores da tabela acima foram lidos do rastro produzido pela execução, não preenchidos apenas como expectativa.

6. Programa 2: um laço de verdade

Objetivo: somar 3 + 2 + 1 = 6. R2 é o contador, R4 acumula a soma e R3 contém 255, padrão de bits que representa -1 em complemento de dois com 8 bits. RAM inicial: M[0]=1, M[1]=3 e M[3]=255.

PC	Hex	Instrução	Finalidade
0	41	LD R1,[R0]	R1 = 1
1	4A	LD R2,[R1]	R2 = 3
2	53	LD R3,[R2]	R3 = 255 (-1)
3	D3	BEQZ R2,+3	Se contador=0, ir ao PC 7
4	14	ADD R4,R2	Acumula o contador
5	1A	ADD R2,R3	Decrementa: soma -1 em 8 bits
6	C4	BEQZ R0,-4	Volta ao PC 3
7	8C	ST R4,[R1]	Grava 6 em M[1]
8	C7	BEQZ R0,-1	Permanece no PC 8

6.1. Por que o decremento funciona?

```
R2 = 3; R3 = 255
3 + 255 = 258 -> guardamos somente 8 bits -> 2
2 + 255 = 257 -> 1
1 + 255 = 256 -> 0
```

A soma descarta o que ultrapassa 8 bits: isso é aritmética módulo 256. Não há exceção de overflow nem sinal de carry armazenado nesta versão.

Momento	R2: contador	R4: acumulador
Depois da carga	3	0
Após 1ª iteração	2	3
Após 2ª iteração	1	5
Após 3ª iteração	0	6

No PC 3, +3 significa 3+1+3=7. No PC 6, -4 significa 6+1-4=3. O mesmo desvio condicional é não tomado enquanto R2>0 e tomado quando R2=0. Resultado executado: R4=6, M[1]=6 e PC=8 após 18 ciclos, incluindo uma iteração do laço final.

7. Verificação e arquivos entregues

Executado nesta entrega
19 testes automatizados do modelo Python aprovados. Os dois programas de demonstração foram montados e executados, com logs e rastros de estado preservados.

Verificação	Cobertura executada
Soma de 8 bits	65.536 pares de operandos, em dois registradores distintos
Leitura/escrita da RAM	65.536 pares endereço/valor: ST seguido de LD
Desvios	8.192 casos: 256 PCs x 8 offsets x valores 0, 1, 128 e 255
Codificação	Todos os 256 bytes: desmontagem e remontagem
Referência de transição separada	8.192 estados pseudoaleatórios, 32 por byte de instrução
Casos de borda	R0, pausa, reset, PC circular, alias de registradores e rejeição de assembly inválido

Esses testes são evidência funcional, não prova formal de todos os estados. O modelo não simula atrasos elétricos, metastabilidade, frequência máxima ou uso de recursos de FPGA.

VHDL: estado diferente do simulador
O pacote inclui core VHDL-2008, integração de memórias e testbench propostos. Eles NÃO foram compilados, simulados ou sintetizados nesta entrega. GHDL não estava disponível e a instalação não foi concluída. Portanto, a equivalência Python/RTL ainda precisa ser verificada.

7.1. Comandos para repetir a verificação

```
python3 -m unittest discover -s tests -v
python3 src/mini8.py examples/laco.asm \
  --data examples/laco_data.json --steps 18
```

Para o VHDL, com GHDL instalado, sh run_rtl.sh executa análise, elaboração e simulação em VHDL-2008 e solicita um arquivo VCD. Esses comandos seguem a documentação oficial do GHDL. [2]

Pasta	Conteúdo
src/	Montador/simulador Python e proposta VHDL
tests/	19 testes Python e testbench VHDL proposto
examples/	Dois programas e imagens iniciais de RAM
results/	Logs reais, rastros CSV e ROMs em hexadecimal
figures/ e docs/	Diagramas editáveis, PDF e Word do projeto

8. Limites, avaliação e fontes

8.1. O que ficou de fora, conscientemente

Decisão	Consequência para avaliação
Somente quatro opcodes	Sem ADDI, SUB dedicado, operações lógicas, multiplicador ou HALT. Constantes vêm da RAM.
Dois campos de 3 bits	ADD reutiliza o destino como operando. LD/ST não incluem deslocamento.
BEQZ com alcance curto	Desvio relativo -4..+3 bytes; programas maiores exigem organização ou revisão da ISA.
R0 fixo em zero	Facilita endereço zero e salto incondicional, mas restam sete registradores graváveis.
Sem periféricos ou exceções	A observação ocorre por simulação/depuração. Não é um produto comercial pronto.

8.2. Limite de implementação física

CPI=1 é uma escolha do modelo com memórias de leitura combinacional. Memórias com leitura registrada exigem rever esse contrato. Como exemplo específico de fabricante, a AMD indica RAM distribuída quando se exige leitura assíncrona na família documentada em UG574. Isso não seleciona uma placa para o Mini-8. [3]

Não atribuímos 125 MHz ou 8 ns a este projeto: esses números pertenciam ao exercício MIPS do slide 13. [1] O período de 10 ns escrito no testbench é apenas estímulo de simulação; não mede Fmax. Faltam compilação RTL, equivalência, síntese, análise temporal e teste em hardware.

8.3. Critérios para nossa avaliação

Verificar se as quatro instruções são suficientes para o objetivo didático; se a codificação fica clara; se o datapath executa a semântica especificada; e se as limitações justificam mudar a ISA antes de escolher uma placa. Esta versão está pronta para essa avaliação arquitetural, não para ser anunciada como hardware validado.

8.4. Referências e rastreabilidade

[1] ROCHA, Herbert Oliveira. Unidade de Controle MIPS Uniciclo. Material fornecido pelo usuário. Slide 14: proposta e entregáveis; slides 6-9: conceitos de datapath/controle; slide 13: exemplo de temporização. Sem data no arquivo.

[2] GHDL. Simulation; Invoking GHDL. Documentação oficial consultada em 02/11/2026. https://ghdl.github.io/ghdl/quick_start/simulation/index.html ; https://ghdl.github.io/ghdl/using/InvokingGHDL.html

[3] AMD. UG574: Distributed RAM Applications. Rev. 1.6, 22/01/2025. Consulta: 02/11/2026. https://docs.amd.com/r/en-US/ug574-ultrascale-clb/Distributed-RAM-Applications

ISA Mini-8, diagramas, exemplos, códigos e testes: elaboração desta proposta. Nenhuma dessas escolhas adicionais é atribuída ao professor.